Experiment-8

Write A Python Program To Implement DFS

Aim

To write and execute a Python program to implement Depth-First Search (DFS) for traversing a graph by exploring each branch as deeply as possible before backtracking.

Algorithm

1. Start the program.
2. Represent the graph using an adjacency list.
3. Select the starting node.
4. Initialize an empty set to store visited nodes.
5. Mark the starting node as visited.
6. Display the current node.
7. Get the neighboring nodes of the current node.
8. Check whether each neighboring node has been visited.
9. If a neighbor is unvisited, recursively call DFS for that node.
10. Continue exploring until no unvisited neighbors remain.
11. Backtrack to the previous node and explore other unvisited neighbors.
12. Stop when all reachable nodes have been visited.

Code

ex-8.py - D:/ex-8.py (3.10.0)

File Edit Format Run Options Window Help

```
# Program to implement DFS

def dfs(graph, start, visited=None):
    if visited is None:
        visited = set()

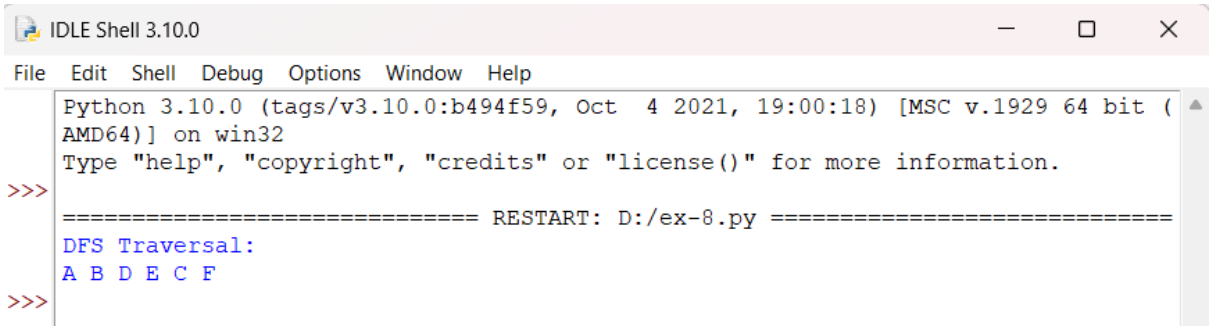
    visited.add(start)
    print(start, end=" ")

    for neighbor in graph[start]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited)


# Graph representation
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}

# Start DFS from node A
print("DFS Traversal:")
dfs(graph, 'A')
```

Output



Result

The Depth-First Search (DFS) algorithm was successfully implemented . The program traverses the graph starting from node A and produces the output: